

Lekcja 08: Flappy

Cel

Ptak, grawitacja, rury. Nauczysz się **struktur** – łączenia kilku zmiennych w jedną rzecz.

Co nowego

Ptak ma `x`, `y` i prędkość `vy`. Rura ma `x` i wysokość przerwy. Zamiast sześciu osobnych zmiennych (`bird_x`, `bird_y`, `bird_vy`, `pipe_x` ...) opisujesz **typ** i tworzysz zmienne tego typu:

```
struct Bird {           // nowy typ: Bird składa się z trzech int
    int x;
    int y;
    int vy;
};                       // uwaga na średnik!

Bird bird;              // zmienna typu Bird
bird.y = 120;           // do pola dostajesz się przez kropkę
bird.vy = bird.vy + 1;
```

Struktura zachowuje się jak zwykła zmienna: przekażesz ją do funkcji (`draw_pipe(pipe)`), skopiujesz (`Pipe p2 = pipe;`), a w następnej lekcji zrobisz z niej tablicę (`Pipe pipes[3]`). W Scratchu nie ma odpowiednika – to jeden z powodów, dla których „prawdziwe” języki są wygodniejsze przy większych grach.

Grawitacja w liczbach całkowitych: co `GRAVITY EVERY` klatek `vy` rośnie o 1, a `y` co klatkę rośnie o `vy`. Machnięcie skrzydłami ustawia `vy` na wartość ujemną (w górę).

Uruchom

Otwórz `gra.cpp`, **Ctrl+Shift+B**. Żółty ptak spada, **Z** (klawisz A) podbija go. Zielona rura z przerwą jedzie w lewo, za ekranem pojawia się nowa z losową przerwą. Zderzenia jeszcze nie ma – ptak przelatuje przez rury i spada pod ziemię.

Jak to działa

```
if (frame_count() % GRAVITY EVERY == 0) bird.vy = bird.vy + 1; // grawitacja: coraz szybciej
w dol
if (pressed(A)) bird.vy = FLAP;                                // skrzydła: nagle w gore
bird.y = bird.y + bird.vy;                                       // ruch
pipe.x = pipe.x - PIPE_SPEED;                                    // rura w lewo
if (pipe.x + PIPE_W < 0) new_pipe();                             // za ekranem -> nowa
```

`draw_pipe(Pipe p)` rysuje dwie części: od góry do `gap_y` i od `gap_y + GAP` do dołu. Między nimi przerwa.

Zadania

1. ★ Zmień `GAP` (łatwiej/trudniej), `FLAP`, kolor ptaka. Spróbuj `GRAVITY_EVERY = 1` – jak zmienia się „ciężar” ptaka?
2. ★★ **Zderzenia**. Dodaj `bool dead`. Ptak umiera, gdy: `bird.y + BIRD_R > screen_height()` (ziemia), albo dotyka rury: `overlaps(bird.x - BIRD_R, bird.y - BIRD_R, 2 * BIRD_R, 2 * BIRD_R, pipe.x, 0, PIPE_W, pipe.gap_y)` (górna część) lub tak samo z dolną częścią. Gdy `dead` – zatrzymaj wszystko, „GAME OVER”, `restart()` po A. `watch("dead", dead);` . (test 1)
3. ★★ **Punkt** za przelecenie rury. Dodaj do `struct Pipe` pole `bool passed;`. W `new_pipe()` ustaw `false`. Gdy `pipe.x + PIPE_W < bird.x && !pipe.passed - score++`, `pipe.passed = true`. Sprawdź, że punkt wpada raz.
4. ★★ **Druga rura**. Zmienna `Pipe pipe2;` startująca w połowie drogi (`pipe2.x = screen_width() + screen_width() / 2`). Zauważ, jak dużo kodu trzeba skopiować (ruch, rysowanie, zderzenie, punkt). W następnej lekcji załatwi to tablica struktur.
5. ★★★ Ptak z charakterem: zamiast koła narysuj funkcją `draw_bird(Bird b)` ciało, oko i dziób, a gdy `b.vy < 0`, skrzydło w górze (animacja). Do tego chmury w tle: 3 struktury `Cloud {int x; int y;}` jadące wolniej niż rury (paralaksa).

Sprawdź sam

Po zadaniu 2, bez klawiszy, ptak spada i uderza w ziemię przed 200. klatką:

```
console_sim.exe --game flappy --frames 200 --trace 40
```

W ostatniej linii `dead=true`.

Dla ciekawych

Dlaczego `struct` a nie sześć zmiennych? Spróbuj dodać do gry trzeciego ptaka (dwóch graczy). Z `Bird bird2;` to jedna linia i kilka `bird2.` – bez struktur musiałbyś wymyślać `bird2_x`, `bird2_y`, `bird2_vy` i pilnować, żeby nigdzie ich nie pomylić.

Słowniczek

po polsku	w C++
nowy typ z kilku pól	<code>struct Nazwa { int a; int b; };</code>
zmienna tego typu	<code>Nazwa z;</code>
pole struktury	<code>z.a</code>
struktura jako parametr	<code>void f(Nazwa z)</code>
kopia struktury	<code>Nazwa z2 = z;</code>